

Environment Setup & Survival Guide

Build a working Boto3 workspace on AWS Academy Learner Lab, and keep it working session after session.

TYPE Setup & reference	REGION us-east-1	SESSION 4 hours	KEY PAIR / ROLE vockey / LabRole
--------------------------------------	----------------------------	---------------------------	--

1 Purpose

This sheet prepares the workspace used by every other lab in the series. It shows how to start a Learner Lab session, hand its temporary credentials to Boto3, set up Python, and work inside the limits of the platform. Keep it open while you work: it answers most of the questions that come up during the module.

What you will be able to do

- Start a Learner Lab session and load fresh credentials into your machine.
- Verify the connection with a call that needs no permissions.
- Prepare an isolated Python environment with Boto3.
- Recognise the limits of the Learner Lab before they block you.
- Translate the most common AWS error codes into a fix.
- Finish every session with a complete cleanup.

2 Starting a session (every time)

1. Open the Learner Lab and click **Start Lab**. Wait until the indicator next to AWS turns green.
2. Click **AWS Details**, then **Show** beside AWS CLI to reveal the credentials block.
3. Replace the whole content of `~/.aws/credentials` with that block.
4. Run the connection test (section 3) before writing any other code.

Credentials are temporary

They change every time you press Start Lab and expire when the 4-hour session ends. Re-copy all three lines at the start of every session. This is expected behaviour, not a bug.

3 Configuration files and connection test

~/.aws/credentials

On Windows the file is `C:\Users\<you>\.aws\credentials`, with no file extension.

```
[default]
aws_access_key_id      = ASIA.....
aws_secret_access_key  = wJalrXUtnFEMI/K7MDENG/.....
aws_session_token      = IQoJb3JpZ2lux2VjE.....
```

~/.aws/config

```
[default]
region = us-east-1
output = json
```

Connection test

```
import boto3

sts = boto3.client("sts")
print(sts.get_caller_identity())
print("Region:", boto3.session.Session().region_name)
```

Why this call?

`get_caller_identity()` requires no permission at all. If it fails, the problem is your credentials, never your IAM policy.

4 Python environment (once per machine)

TERMINAL

```
python --version          # 3.9 or newer

python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate

pip install boto3
pip freeze > requirements.txt
```

5 What the Learner Lab allows

Area	Limit	What to do
Regions	us-east-1 and us-west-2 only	Use us-east-1 for everything in this module.
EC2 types	nano, micro, small, medium, large	Use t3.micro . Larger types fail with UnauthorizedOperation .
EC2 count	9 instances, 32 vCPU per region	Launch one at a time and terminate when finished.
EBS volumes	100 GB maximum	The default 8 GB root volume is enough.
IAM	No users, groups or roles can be created	Use the pre-existing LabRole .
Key pairs	vockey already exists in us-east-1	Pass KeyName="vockey" to run_instances .
Session	4 hours, then everything stops	Press Start Lab again to reset the timer.
Budget	Fixed, shared credit	Terminate everything: idle resources burn credit for nothing.

6 Lambda execution role

A Lambda function normally needs an execution role, but the Learner Lab does not let you create IAM roles. Use the existing **LabRole** instead, and build its ARN from your own account ID rather than typing it.

```
sts      = boto3.client("sts")
account  = sts.get_caller_identity()["Account"]
role_arn = f"arn:aws:iam::{account}:role/LabRole"

lambda_client.create_function(
    FunctionName=name,
    Runtime="python3.13",
    Role=role_arn,          # <- the only line that differs in the lab
    Handler="lambda_function.lambda_handler",
    Code={"ZipFile": package_bytes},
)
```

7 Error codes, translated

Error code	What it means	Fix
Credentials and configuration		
<code>ExpiredToken</code>	The session token has run out.	Press Start Lab and re-copy all three credential lines.
<code>ExpiredTokenException</code>	Same problem, reported by another service.	Same fix.
<code>InvalidClientTokenId</code>	The access key is not valid here.	You pasted a partial or stale block. Re-copy it.
<code>NoRegionError</code>	Boto3 found no region at all.	Add <code>region = us-east-1</code> to <code>~/.aws/config</code> .
Permissions		
<code>AccessDenied</code>	Valid identity, missing permission.	Read the message: it names the exact action.
<code>UnauthorizedOperation</code>	The EC2 flavour of <code>AccessDenied</code> .	Usually a forbidden instance type or region.
S3		
<code>NoSuchBucket</code>	The bucket does not exist here.	Check the name, then the region.
<code>BucketAlreadyExists</code>	Another AWS account owns that name.	Bucket names are global. Add your own prefix.
<code>BucketNotEmpty</code>	The bucket still contains objects.	Delete every object first, then the bucket.
Other services		
<code>ResourceNotFoundException</code>	Wrong name, or wrong region.	Print <code>Session().region_name</code> and check.
<code>ResourceInUseException</code>	The table or function already exists.	Use it, or delete it first.
<code>ConditionalCheckFailedException</code>	Your DynamoDB condition held.	Not a crash: handle it and inform the user.
<code>ValidationException</code>	Malformed parameters.	Re-read the documentation of that operation.
<code>ThrottlingException</code>	You are calling too fast.	Let botocore retry; use adaptive retry mode.

8 Four patterns you will reuse

Paginator: AWS never returns everything at once

```
paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(Bucket=name):
    for obj in page.get("Contents", []):      # .get: an empty page has no such key
        print(obj["Key"])
```

Waiter: AWS answers before the work is finished

```
ec2.get_waiter("instance_running").wait(
    InstanceIds=[instance_id],
    WaiterConfig={"Delay": 15, "MaxAttempts": 40},
)
```

Error codes: branch on the code, never on the message

```
from botocore.exceptions import ClientError

try:
    ...
except ClientError as e:
    code = e.response["Error"]["Code"]
    if code == "NoSuchBucket":
        print("That bucket does not exist.")
    else:
        raise # never swallow an unknown error
```

Logging and configuration

```
import logging, os

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)-8s %(message)s")
log = logging.getLogger("toolkit")

REGION = os.environ.get("AWS_REGION", "us-east-1")
```

9 End-of-session cleanup

Run through this list before you close your laptop. Every session, without exception.

- [] EC2: every instance you launched is **terminated**, not just stopped.
- [] S3: every bucket you created is emptied and deleted.
- [] DynamoDB: every table you created is deleted.
- [] Lambda: every function you deployed is deleted.
- [] The AWS Console shows nothing left that belongs to you.
- [] Your work is committed and pushed to your Git repository.

Stopped is not cleaned up

A stopped EC2 instance costs no compute, but its EBS volume is billed for as long as it exists. Only terminating the instance stops every charge.

Golden rule

If you created it, you destroy it. Cleanup is half the job in cloud automation.